C++ User Input

Mastering Interactivity with cin

CodeHive Academy

Building Modern, Interactive Software

1. The concept of Input

In your previous CodeHive lessons, you learned that `cout` is the primary tool for **Output**—pushing data from your program to the user's screen. However, for a program to be truly useful, it must be able to **listen** to the user.

What is cin?

- **Name:** `cin` stands for "Console Input".
- **Source:** It reads data directly from the keyboard.
- **Stream:** It lives within the `<iostream>` library.

> **The Flow:** While `cout` sends data OUT (<<), `cin` pulls data IN (>>).

The Extraction Operator (>>)

The >> sign is called the Extraction Operator. Think of it as a funnel that extracts the characters from the keyboard and pours them into your variable container.

2. Basic Syntax & Workflow

To capture input at CodeHive, follows these three logical steps:

1. **Declare:** Create a variable to hold the incoming data.
2. **Prompt:** Use `cout` to tell the user what you need.
3. **Extract:** Use `cin` to capture the response.

```
int x; cout << "Type a number: "; cin >> x; cout << "Your number is:
" << x;
```

```
Type a number: 42
Your number is: 42
```

Notice: We do not use `endl` on the prompt line so the user can type their answer right next to the question.

3. Comparison: cout vs. cin

It is easy to mix these up when starting your CodeHive journey. Use this reference table to stay on track.

Term	Full Name	operator	Direction
cout	See-Out	<< (Insertion)	Program → Screen
cin	See-In	>> (Extraction)	Keyboard → Variable

Memory Link: The arrows always point in the direction the data is moving.

- **Out** moves to the cout object.
- **In** moves from the cin object to your variable.

4. Building a Simple Calculator

Let's combine everything we've learned to build a functional addition calculator for CodeHive.

```
#include <iostream> using namespace std; int main() { int x, y, sum;
cout << "Type the first number: "; cin >> x; cout << "Type the second
number: "; cin >> y; sum = x + y; cout << "The final sum is: " << sum
<< endl; return 0; }
```

```
Type the first number: 5
Type the second number: 7
The final sum is: 12
```

5. Handling Textual Input

Variables aren't just for numbers. At CodeHive, we often capture names, cities, or passwords as **Strings**.

```
#include <iostream> #include <string> using namespace std; int main()
{ string name; cout << "Enter your username: "; cin >> name; cout <<
"Welcome, " << name << "!" << endl; return 0; }
```

The cin String Limitation: Standard `cin` is "whitespace sensitive." This means it stops reading as soon as you hit the Space bar, Enter, or Tab.

If you type "CodeHive Academy", `cin` will only store "CodeHive".

6. Full Line Reading (getline)

To overcome the "one-word" limitation of `cin`, C++ provides a specialized function called `getline()`.

Using getline()

This function reads the entire line of text until you press the Enter key, including all spaces.

```
string fullName; cout << "Enter your full name: "; getline(cin, fullName); cout << "Account created for: " << fullName;
```

Syntax Breakdown: `getline(source, destination)`.

- Source is `cin` (keyboard).
- Destination is your variable.

7. Multi-Input Chaining

Just as you can chain `cout <<`, you can chain `cin >>` if you know the exact sequence of items the user will provide.

```
int day, month, year; cout << "Enter date (DD MM YYYY): "; cin >> day
>> month >> year;
```

Input Safety at CodeHive

If you ask for an `int` and the user types "hello", the program will fail to read properly. This is known as an **Input Failure**. In advanced lessons, we will learn how to validate this data.

8. Summary & Practice

Congratulations! You can now build programs that talk back and listen to users. This is the foundation of all software—from games to banking apps.

Key Takeaways:

- `cin >>` is for input.
- `cout <<` is for output.
- `getline()` is for reading spaces in text.
- Data types must match what the user is typing.

🛠 CodeHive Lab Challenges

1. **The Profile Maker:** Ask for a user's nickname and age, then print it back in a single sentence.
2. **The Product:** Similar to our Sum calculator, take two numbers and print their **Product** (multiplication).
3. **The Full Address:** Use `getline()` to capture a user's street address and display it.
